GST Reconciliation Engine

Knowledge Graph-Powered ITC Validation & Fraud Detection

Tech Stack

Backend: Python 3.12 · FastAPI · Neo4j (Bolt/Aura) · scikit-learn · Pydantic v2

Frontend: React 18 + TypeScript · Vite · TanStack Query · Recharts · Tailwind CSS

ML: IsolationForest · RandomForestClassifier · MinMaxScaler (joblib)

Auth: JWT (HS256) · Clerk (JWKS) · Bearer token middleware

Deliverable 1 — Knowledge Graph schema & GST data model

Deliverable 2 — Reconciliation engine with mismatch classification

Deliverable 3 — ITC risk & vendor compliance dashboard

Deliverable 4 — Explainable audit trail generator

Domain: India's GST ecosystem (GSTR-1/2B/3B, e-Invoice, e-Way Bill)

Problem: ITC leakage affecting 1.4 crore taxpayers

Approach: Graph traversal, not flat-table matching

Scale: 50 000+ invoices, batch + on-demand reconciliation

Contents

1. Problem Statement & Motivation

India's Goods and Services Tax (GST) system generates billions of invoice records every month across the GSTR-1 (outward supplies), GSTR-2B (auto-drafted inward), and GSTR-3B (summary tax payment) returns. **Input Tax Credit (ITC)** allows buyers to claim credit for the GST paid by their suppliers, but this mechanism is vulnerable to:

- **Fake invoicing:** Suppliers file GSTR-1 for non-existent sales to fraudulently enable ITC claims by connected buyers.
- **Value mismatches:** Supplier-reported and buyer-recorded amounts differ beyond acceptable tolerances.
- **Circular trading:** A chain $A \to B \to C \to A$ of invoices inflates turnover and GST credits artificially.
- **Late/missing filings:** Gaps between invoice date, GSTR-1 filing, and tax payment timing indicate evasion.
- **Amendment abuse:** Repeatedly amending invoices to alter taxable values after filing.

Core Insight

GST reconciliation is fundamentally a **graph traversal problem**. Every invoice connects a seller (Taxpayer) to a buyer (Taxpayer) through a chain of filings (GSTR-1 → GSTR-2B) and payments (TaxPayment → GSTR-3B). Traditional flat-table JOIN-based matching misses cross-entity anomalies that are only visible when the full transaction network is modelled as a graph.

2. System Architecture

2.1. High-Level Architecture

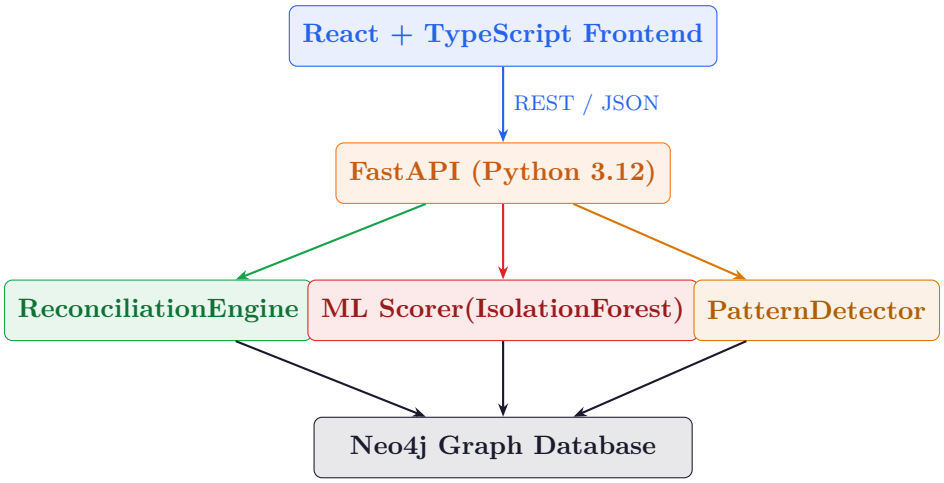

Figure 1: System architecture: React frontend communicates with a FastAPI backend that orchestrates three service layers (Reconciliation, ML, Patterns) all backed by a shared Neo4j graph database.

2.2. Request Lifecycle

1. User uploads CSV files (taxpayers, invoices, GSTR-1/2B/3B, payments) via the React Upload wizard.

2. FastAPI parses and validates each row with Pydantic v2 schemas, then calls the `graph_builder` service.

3. `graph_builder` issues idempotent Cypher `MERGE` statements to Neo4j, constructing nodes and relationships.

4. The Reconciliation Engine traverses invoice subgraphs: path validation → value comparison → timing checks → plain-English explanation, writing results back as node properties.

5. The ML Scorer extracts an 8-dimensional feature vector per vendor from Neo4j, runs `IsolationForest` anomaly detection, computes a 0–100 compliance score, and persists `risk_score` on `Taxpayer` nodes.

6. The Pattern Detector runs four independent Cypher queries to find circular trades, payment delays, amendment chains, and high-risk networks.

7. The frontend polls these results via TanStack Query, rendering charts (Recharts), a force-layout graph explorer, vendor compliance gauges, and pattern detection tables.

3. Knowledge Graph Schema

3.1. Node Labels

Label	Key Properties	Primary Key
`Taxpayer`	gstin, pan, state_code, registration_status, filing_frequency, risk_score	gstin
`Invoice`	invoice_number, invoice_date, gstr1_taxable_value, pr_taxable_value, cgst, sgst, igst, total_value, status, risk_level, explanation	invoice_id
`GSTR1`	gstin, tax_period, filing_date, status	return_id
`GSTR2B`	gstin, tax_period, generation_date	return_id
`GSTR3B`	gstin, tax_period, filing_date, tax_payable, tax_paid	return_id
`TaxPayment`	amount_paid, payment_date, payment_mode	payment_id
`EInvoice`	ack_date, signed_hash	irn
`EWayBill`	generation_date, distance, vehicle_no	ewaybill_no

Table 1: Knowledge Graph node labels and their primary key properties. All primary keys have `CREATE CONSTRAINT ... IF NOT EXISTS` uniqueness enforcement applied at startup.

3.2. Relationship Types

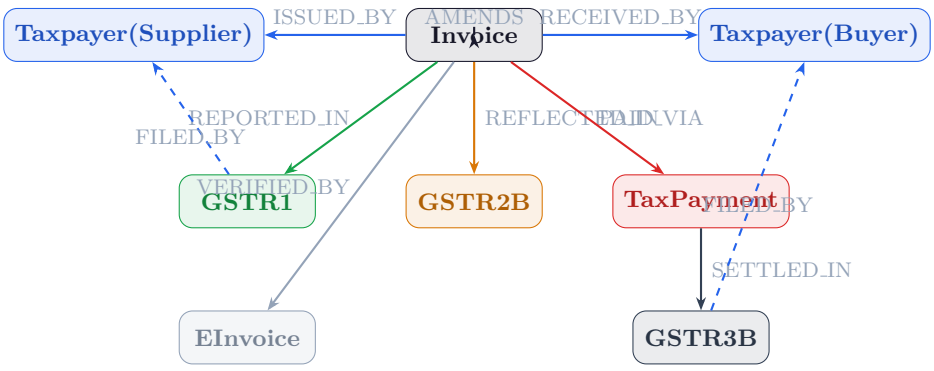

Figure 2: Invoice-centric relationship graph. Solid arrows show primary data relationships. Dashed arrows are secondary (filing-party) relationships used for multi-hop traversal. The `AMENDS` self-loop on `Invoice` tracks amendment versioning chains.

3.3. Design Decisions

- **Idempotent ingestion:** All writes use Cypher `MERGE` + `SET += {...}`, so re-uploading the same file is safe and does not duplicate data.
- **Trust hierarchy:** When both e-Invoice IRN and GSTR-1 values exist, e-Invoice is treated as the authoritative source. GSTR-1 takes precedence over the Purchase Register (PR).
- **Computed node properties:** `status`, `risk_level`, and `explanation` on `Invoice` nodes, and `risk_score` on `Taxpayer` nodes, are *not* stored during ingestion — they are computed by the Reconciliation Engine and ML Scorer and written back afterwards, keeping ingestion and analysis cleanly separated.
- **Indexes:** Nine non-primary-key indexes (e.g. `Invoice.status`, `Taxpayer.risk_score`, `GSTR1.tax_period`) accelerate the most frequent dashboard queries.

4. Data Ingestion Pipeline

4.1. Upload Flow

The **React Upload wizard** guides users through four sequential steps matching the upload dependency order:

Step 1: **Taxpayers** (GSTIN master) → Step 2: **Invoices** (B2B records) → Step 3: **GST Returns** (GSTR-1 / 2B / 3B) → Step 4: **Tax Payments** (challan ledger)

Each zone supports drag-and-drop or file-picker CSV upload. The backend endpoint `POST /upload/{type}` streams rows through:

1. **Pydantic parsing** — columns are mapped to typed ingestion row models (`TaxpayerIngestionRow`, `InvoiceIngestionRow`, etc.) with field validators.
2. **Batch UNWIND** — batches of up to 500 rows are sent as a single Cypher `UNWIND $batch AS r MERGE ...` statement for throughput.
3. **Response:** `{"loaded": N, "skipped": K}` returned to the UI.

4.2. Synthetic Data Generator

A dedicated `backend/data/synthetic/generator.py` creates realistic test data with injected anomalies (`AnomalyType` enum: `NONE`, `MISSING_PAYMENT`, `VALUE_MISMATCH`, `CIRCULAR_TRADE`, `LATE_FILING`, `AMENDMENT`) enabling end-to-end testing of the reconciliation and pattern detection pipelines without real taxpayer data.

5. Reconciliation Engine

The reconciliation engine (`services/reconciliation/`) is the core analytical component. It evaluates every invoice through three independent checkers and combines their results into a final verdict.

5.1. Three-Checker Architecture

5.1.1. Path Validator (`path_validator.py`)

Checks whether all five expected relationships exist for an invoice:

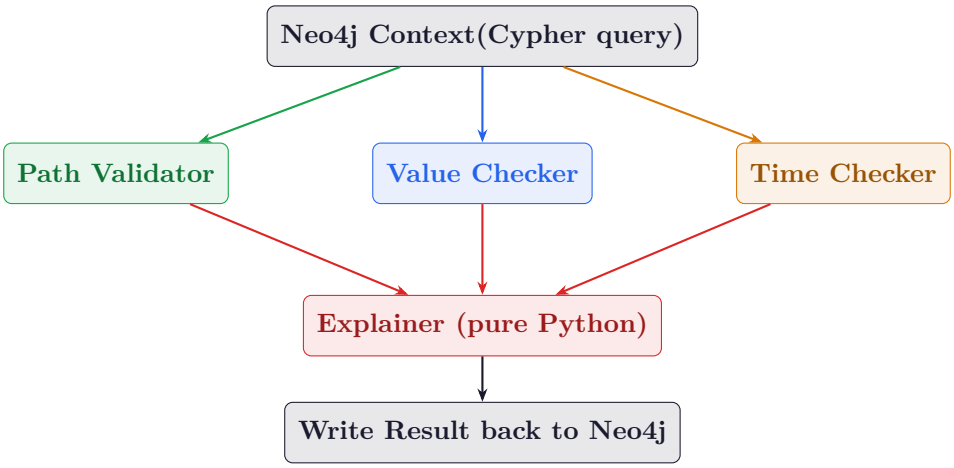

Figure 3: Reconciliation engine flow for a single invoice. Context is fetched once from Neo4j and fanned out to three pure-Python checkers. The Explainer combines their results into status, risk level, and a natural-language explanation.

Relationship	Meaning	Missing Impact
`ISSUED_BY`	Supplier Taxpayer linked	HIGH risk flag
`RECEIVED_BY`	Buyer Taxpayer linked	Warning flag
`REPORTED_IN`	Filed in GSTR-1	HIGH risk flag
`REFLECTED_IN`	Visible in buyer GSTR-2B	Warning flag
`PAID_VIA`	Tax payment exists	Warning flag

Table 2: Path check results and their risk impact. Coverage score = number of present relationships (max 5).

5.1.2. Value Checker (`value_checker.py`)

Cross-checks the taxable value declared in GSTR-1 against the Purchase Register (PR):

$$\text{deviation\_pct} = \frac{|\text{gstr1\_val} - \text{pr\_val}|}{\text{gstr1\_val}} \times 100$$

- deviation_pct $\leq 2\%$ (configurable) $\rightarrow$ **within tolerance**
- $2\% <$ deviation_pct $\leq 10\% \rightarrow$ **Warning**
- deviation_pct $> 10\% \rightarrow$ **HIGH RISK**

Also verifies internal GST math: CGST + SGST + IGST $\approx$ taxable_value $\times$ blended_rate.

5.1.3. Time Checker (`time_checker.py`)

Evaluates two timing windows:

- **Filing delay:** GSTR-1 must be filed by the 11th of the month following the invoice period. >30 days late $\rightarrow$ HIGH risk.
- **Payment delay:** `payment_date` vs `invoice_date`. Chronic delay (configurable threshold) $\rightarrow$ HIGH risk.

5.1.4. Explainer — Status Decision Matrix

Status	Risk Level	Condition
`VALID`	Low	All paths present, values within tolerance, no timing issues
`WARNING`	Medium	Minor value deviation, GSTR-2B absent, late filing ≤30 days, or moderate payment delay
`HIGH_RISK`	High	Value deviation >10%, GSTR-1 missing, both supplier and payment absent, or chronic delay
`PENDING`	—	No GSTR-1 and no PR value (no source data at all)

Table 3: Status decision matrix used by the Explainer. Results are written to `Invoice.status`, `Invoice.risk_level`, and `Invoice.explanation` in Neo4j.

5.2. Batch & On-Demand Reconciliation

- **`POST /reconcile/run`** — batch mode; optionally filtered by `gstin` and/or `tax_period` (`MMYYYY`), with a configurable `limit` (up to 50 000).
- **`POST /reconcile/invoice/{id}`** — on-demand single invoice reconciliation.
- **`GET /reconcile/stats`** — returns live status distribution counts without reprocessing.

6. ML-Based Vendor Risk Scoring

6.1. Feature Engineering

Eight features are extracted per vendor directly from Neo4j using Cypher aggregation queries:

#	Feature	Description
0	`invoice_count`	Total invoices issued as seller
1	`high_risk_ratio`	Fraction of invoices with status = High-Risk
2	`warning_ratio`	Fraction of invoices with status = Warning
3	`amendment_rate`	Fraction of invoices with an `AMENDS` edge
4	`missing_payment_rate`	Fraction of invoices with no `PAID_VIA` link
5	`avg_delay_days`	Average (payment_date − invoice_date) in days
6	`late_filing_rate`	Fraction of GSTR-1 filings submitted after the 11th
7	`value_mismatch_rate`	Fraction of invoices flagged as VALUE_MISMATCH

Table 4: 8-dimensional vendor feature vector. Extracted by three Cypher aggregation queries and combined in Python.

6.2. Compliance Score Formula

The rule-based score applies weighted penalties:

$$\text{penalty} = 35 \cdot r_{\text{high}} + 15 \cdot r_{\text{warn}} + 20 \cdot r_{\text{late}} + 10 \cdot r_{\text{missing\_pay}} + 10 \cdot r_{\text{amend}} + 5 \cdot r_{\text{mismatch}} + 5 \cdot \min\left(\frac{d_{\text{avg}}}{90}, 1\right)$$

$$\text{score} = \max(0,\ \min(100,\ 100 - \text{penalty}))$$

An optional **IsolationForest** anomaly deduction (0–5 pts) is added when the model is available, providing an unsupervised boost for vendors whose feature vectors lie far from the normal distribution.

6.3. Risk Level Thresholds

score $\geq 75 \rightarrow$ **Low Risk** $\quad 50 \leq$ score $< 75 \rightarrow$ **Medium Risk** $\quad$ score $< 50 \rightarrow$ **High Risk**

6.4. Model Training

Unsupervised: `IsolationForest` (contamination = 0.1) detects anomalous vendors without labels.

Supervised: `RandomForestClassifier` is trained when ≥ 20 labelled samples are available. Labels are derived automatically from the graph: a vendor is "risky" if $\geq$30% of their invoices have `status` $\in$ {`High-Risk, Warning`}.

Both models are persisted with `joblib` to `data/models/` alongside a `feature_meta.json` containing feature names and training timestamp. The API exposes `POST /vendors/train` and `POST /vendors/score` to trigger these steps from the frontend.

7. Fraud Pattern Detection

Four Cypher-powered pattern detectors run on-demand via `GET /patterns/all`:

7.1. Circular Trade Detection

The most sophisticated pattern: finding closed trading loops $A \rightarrow B \rightarrow C \rightarrow A$ that indicate round-tripping to inflate ITC claims.

Simplified 3-node Cypher query

```
MATCH
    (i1:Invoice)-[:ISSUED_BY]->(a:Taxpayer),
    (i1)-[:RECEIVED_BY]->(b:Taxpayer),
    (i2:Invoice)-[:ISSUED_BY]->(b),
    (i2)-[:RECEIVED_BY]->(c:Taxpayer),
    (i3:Invoice)-[:ISSUED_BY]->(c),
    (i3)-[:RECEIVED_BY]->(a)
WHERE a <> b AND b <> c AND a <> c
  AND a.gstin < b.gstin AND a.gstin < c.gstin
RETURN a.gstin, b.gstin, c.gstin, i1.invoice_id, i2.invoice_id, i3.invoice_id
```

Loops are canonicalised (sorted GSTIN tuple $\rightarrow$ MD5 hash) to deduplicate rotational variants. Both 2-node (back-and-forth) and 3-node loops are detected.

7.2. Payment Delay Detection

Calculates average payment delay per vendor and flags:

- **High risk** when avg_delay $\geq$ `CHRONIC_DELAY_DAYS` (configurable)
- **Medium risk** when avg_delay $\geq$ `WARN_DELAY_DAYS`

7.3. Amendment Chain Detection

Walks `(new_inv)-[:AMENDS*1..5]->(root:Invoice)` chains up to depth 5. Vendors are flagged when:

- `chain_count` $\geq$ `AMENDMENT_FLAG_COUNT` $\rightarrow$ HIGH risk
- Any chain depth $\geq 2 \rightarrow$ HIGH risk (repeated amendments to a single invoice)

7.4. Risk Network Analysis (Guilt-by-Association)

For each taxpayer, computes:

$$\text{risky\_ratio} = \frac{\text{partners with risk\_level} = \text{'High'}}{\text{total distinct trading partners}}$$

Vendors whose trading networks are predominantly high-risk are flagged even if their own invoices appear clean, implementing a graph-based guilt-by-association heuristic.

8. Frontend Dashboard

8.1. Technology Stack

Library	Purpose
React 18 + TypeScript	Component framework with strong typing
Vite + Bun	Fast build tooling and package management
TanStack Query v5	Server state management, caching, background refetch
React Router v6	SPA client-side routing
Recharts	BarChart (monthly reconciliation) + PieChart (risk distribution)
Framer Motion	Page entry/exit animations
Tailwind CSS + shadcn/ui	Design system, accessible components
Lucide React	Icon library

Table 5: Frontend library choices and their roles.

8.2. Page Overview

`/dashboard` Live stats (total invoices, high-risk count, ITC at risk, valid

`/invoices` Paginated, filterable table of all invoices with status badges (Valid / Warning / High-Risk / Pending), drill-down to single-invoice reconciliation, and one-click batch reconciliation trigger.

`/vendors` Vendor compliance table with colour-coded score bars and trust gauges. Exposes *Train ML Model* and *Score All Vendors* buttons that call the backend ML endpoints. Clicking a vendor shows a detailed profile with invoice history.

`/patterns` Tabbed interface for the four pattern detectors. A *Run Detection* button calls `GET /patterns/all` and renders results in category tables with risk badges.

`/graph` Full-screen interactive force-layout graph explorer. Nodes are colour-coded by type (Taxpayer, Invoice, GSTR1, GSTR2B, GSTR3B, TaxPayment). Supports zoom/pan, search by node ID, and click-to-expand neighbours. Runs a JavaScript force simulation (repulsion + spring) in canvas for performance.

`/upload` Multi-step wizard with drag-and-drop zones per data type, real-time upload status, and navigation guard preventing upload reordering.

8.3. API Client (`src/lib/api.ts`)

A single typed API client handles all backend communication:

- Bearer token from `localStorage` attached to every request.
- A typed `ApiError` class wraps non-2xx responses with the FastAPI `detail` field.

- Query-string parameters are cleanly built via `URLSearchParams` (no manual string concatenation).
- FormData uploads bypass the `Content-Type: application/json` header automatically.

9. API Layer (FastAPI)

9.1. Router Structure

Router File	Prefix	Key Endpoints
`auth.py`	`/auth`	`POST /login`, `GET /me`
`upload.py`	`/upload`	`POST /{type}`
`invoices.py`	`/invoices`	`GET /`, `GET /{id}`, `GET /stats`
`vendors.py`	`/vendors`	`GET /`, `GET /{gstin}`, `POST /train`, `POST /score`
`reconcile.py`	`/reconcile`	`POST /run`, `POST /invoice/{id}`, `GET /stats`
`patterns.py`	`/patterns`	`GET /all`, `GET /circular`, `GET /delays`, `GET /amendments`, `GET /networks`
`graph.py`	`/graph`	`GET /overview`, `GET /vendor/{gstin}`

Table 6: FastAPI router layout. All state-mutating endpoints require a Bearer token.

9.2. Application Lifecycle

On startup (`@asynccontextmanager lifespan`):

1. Verify Neo4j connectivity over the Bolt protocol.
2. Run `init_schema()` — idempotent constraints and indexes creation. If Neo4j is unreachable the warning is logged but the app starts anyway (graceful degradation).
3. Register all routers under their prefix paths.

CORS is configured to allow the React dev server (`localhost:3000` / `5173`) and any configured production origin.

10. Key Engineering Decisions

Graph DB over RDBMS Relational databases require multi-table JOINs for multi-hop traversal. With Neo4j, a 4-hop Cypher query (Taxpayer → Invoice → GSTR1 → TaxPayment → GSTR3B) executes in milliseconds via index-free adjacency, and adding new relationship types doesn't require schema migrations.

Pure-Python checker modules The path, value, and time checkers contain zero database calls — they operate only on the context dict fetched by the engine. This makes them trivially unit-testable (inject a context dict, assert the result) and easy to extend with new rules without touching the data layer.

Idempotent ingestion `MERGE` semantics mean that running the same upload twice produces the same graph state. This is essential for a reconciliation system where the same GSTR-1 data may arrive incrementally or be corrected through amendments.

Separation of ingestion and scoring `status`/`risk_level`/`explanation` on Invoice, and `risk_score` on Taxpayer, are written by the analytical services, not the ingestor. This lets the frontend display "Pending" for new uploads and enables incremental re-scoring without re-uploading data.

Force-simulation graph in canvas The graph explorer implements a custom JavaScript force simulation directly on `<canvas>` rather than using a heavy D3/Cytoscape dependency. This

avoids shipping large libraries for what is primarily a presentation layer feature, and keeps the bundle size lean.

TanStack Query for data freshness All dashboard data has a `staleTime` of 30 seconds. After reconciliation or scoring mutations complete, the relevant query keys are invalidated, triggering automatic background refetch — keeping figures in sync without a full page reload.

11. Security Considerations

- **Authentication:** JWT (HS256) with configurable expiry (default 8 hours). Clerk JWKS integration is available for production single-sign-on.
- **Parameterised Cypher:** All database queries use `$param` placeholders — no string-concatenated Cypher, eliminating Cypher Injection risk.
- **Input validation:** Pydantic v2 enforces type correctness and range constraints at the API boundary before any data reaches the database layer.
- **CORS:** Configured with an explicit allowed-origins list — wildcard `*` is only used in development mode.
- **Credentials:** All secrets (Neo4j password, JWT secret, Clerk keys) are loaded from environment variables via `python-dotenv`; no credentials are hardcoded in committed code paths intended for production.

12. Summary & Deliverable Checklist

#	Deliverable	Implementation
1	Knowledge Graph schema & GST data model	8 node labels, 11 relationship types, uniqueness constraints, 9 indexes — all in `database/schema_init.py`
2	Reconciliation engine with mismatch classification	Path + Value + Time checkers feeding the Explainer in `services/reconciliation/`; 4 status levels, 3 risk levels
3	Interactive ITC risk & vendor compliance dashboard	Full React SPA with live charts, force-layout graph, vendor gauges, and real-time reconciliation trigger
4	Explainable audit trail generator	Plain-English bullet-point explanation written to `Invoice.explanation` in Neo4j, surfaced on invoice detail view
5	Predictive vendor compliance risk model	IsolationForest + RandomForestClassifier pipeline, 8-feature vector, 0–100 compliance score, joblib-persisted models
+	Fraud pattern detection (bonus)	Circular trades (2-node + 3-node), payment delays, amendment chains, risk-network analysis

Table 7: All five project deliverables from the problem statement, plus the additional pattern detection module.

One-liner Summary for Interviewers

We built a **full-stack, graph-native GST reconciliation engine** that ingests GSTR-1/2B/3B CSV data into a **Neo4j knowledge graph**, traverses invoice-to-payment

chains to detect ITC fraud and value mismatches, assigns explainable risk scores to vendors using an **IsolationForest ML pipeline**, and surfaces everything through a **React dashboard** with an interactive graph explorer — replacing brittle flat-table SQL joins with purpose-built graph traversal.